

Chapter 4 – Proposed Framework and Implementation

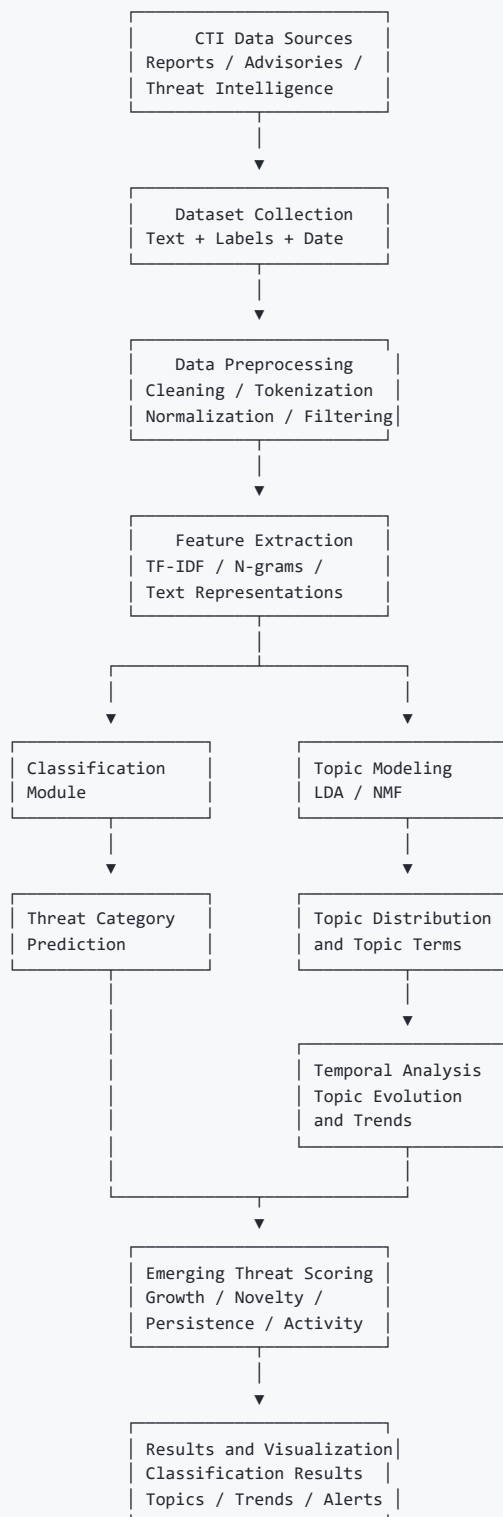
4.1 System Architecture

The proposed framework is designed to automate the analysis of Cyber Threat Intelligence (CTI) reports by combining machine learning-based threat classification with temporal topic modeling and emerging threat detection. The framework processes textual CTI data, extracts meaningful features, classifies threat-related documents, identifies latent topics, analyzes their temporal evolution, and assigns an emerging threat score to potentially significant patterns.

The architecture is organized into several interconnected stages:

1. CTI Data Input
2. Data Preprocessing
3. Feature Extraction
4. Threat Classification
5. Topic Modeling
6. Temporal Analysis
7. Emerging Threat Scoring
8. Results and Visualization

The overall architecture of the proposed framework is represented below.



4.1.1 Architecture Components

The proposed architecture consists of the following major components.

CTI Data Sources

The input layer contains textual cyber threat information obtained from publicly available CTI reports, security advisories, incident reports, vulnerability information, malware reports, and other suitable threat intelligence sources.

Each document should contain, where available, textual information and temporal metadata such as publication date.

Dataset Management Layer

The collected documents are organized into a structured dataset. Important fields may include:

Field	Description
id	Unique document identifier
title	Title of the CTI document
text	Main textual content
category	Threat category or class
date	Publication or observation date
source	Source of the document
url	Reference URL

Preprocessing Layer

The preprocessing component removes irrelevant content and converts raw CTI documents into a consistent representation suitable for machine learning and topic modeling.

Feature Extraction Layer

The cleaned text is transformed into numerical representations. TF-IDF and n-gram features can be used for traditional machine learning models, while other text representations may be evaluated where required.

Classification Layer

The classification module predicts the threat category associated with each CTI document. Multiple machine learning algorithms can be trained and compared.

Topic Modeling Layer

The topic modeling component discovers latent themes from CTI documents without requiring predefined topic labels. LDA and NMF can be used to identify important cybersecurity topics.

Temporal Analysis Layer

Topic distributions are grouped according to time periods to identify changes in threat-related discussions.

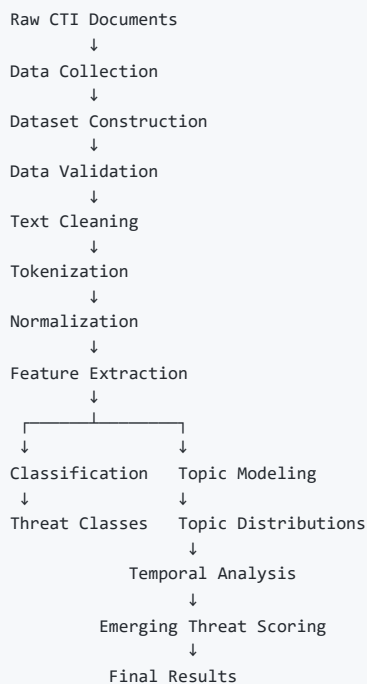
Emerging Threat Scoring Layer

The framework combines temporal and textual signals to identify topics that exhibit characteristics associated with emerging threats.

4.2 Data Pipeline

The data pipeline defines the sequence through which CTI documents are transformed from raw textual information into classification results and emerging threat indicators.

The pipeline consists of the following stages:



4.2.1 Data Collection

The first stage involves collecting CTI documents from selected sources. The collected documents are stored with their associated metadata.

The dataset should maintain the relationship between each document and its publication or observation date because temporal information is essential for emerging threat analysis.

4.2.2 Data Validation

Before preprocessing, the collected dataset is examined for:

- Missing textual content
- Missing dates
- Duplicate documents
- Invalid records
- Extremely short documents
- Incorrect labels
- Inconsistent formatting
- Duplicate URLs
- Unusable or corrupted records

Records that do not satisfy the required quality conditions are removed or corrected where appropriate.

4.2.3 Text Processing

The textual content is cleaned using the preprocessing operations described in Chapter 3. The objective is to reduce irrelevant variation while preserving cybersecurity-specific information.

Particular attention is given to security-related entities such as:

- CVE identifiers
- Malware names
- Vulnerability names
- Attack techniques
- Protocol names
- File extensions
- Domains
- IP addresses
- Security tools
- Threat actor names

Over-aggressive preprocessing can remove information that is useful for CTI analysis. Therefore, preprocessing rules are selected according to the characteristics of the dataset.

4.2.4 Data Representation

After preprocessing, documents are converted into numerical representations.

For traditional machine learning classification, TF-IDF is used to represent the importance of terms within documents.

For topic modeling, the corpus is represented using an appropriate document-term matrix.

4.2.5 Parallel Analytical Processing

After feature extraction, the framework follows two analytical paths.

The first path performs supervised threat classification.

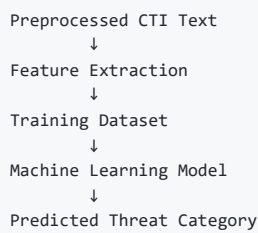
The second path performs unsupervised topic discovery and temporal analysis.

The outputs of both paths are subsequently used for integrated threat analysis.

4.3 Classification Module

The classification module is responsible for automatically assigning CTI documents to predefined threat categories.

The classification process can be represented as:



4.3.1 Classification Objective

Let the training dataset be represented as:

$$D = \{(x_i, y_i)\}_{i=1}^N$$

where:

- x_i represents the feature vector of document i ,
- y_i represents its corresponding threat category,
- N represents the total number of documents.

The classification model learns a function:

$$f: X \rightarrow Y$$

where X represents the document feature space and Y represents the set of predefined threat categories.

For a new CTI document x , the trained model generates:

$$\hat{y} = f(x)$$

where $\hat{y}$ is the predicted threat category.

4.3.2 Classification Models

The framework can evaluate multiple machine learning algorithms, including:

- Naive Bayes
- Logistic Regression
- Support Vector Machine (SVM)
- Random Forest
- Gradient Boosting

The final set of models depends on dataset characteristics and experimental requirements.

4.3.3 Training and Testing

The dataset is divided into training and testing subsets.

A typical configuration may use:

- Training set: 80%
- Testing set: 20%

A stratified split should be considered when the dataset contains class imbalance.

The training data are used to learn the model parameters, while the test data are kept separate for evaluating generalization performance.

4.3.4 Classification Output

For each CTI document, the classification module produces an output similar to:

Document ID	Predicted Category	Confidence/Probability
CTI-001	Malware	0.91
CTI-002	Phishing	0.87
CTI-003	Ransomware	0.94

The actual output format depends on the selected machine learning implementation.

Classification results are evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

4.3.5 Classification and Emerging Threat Analysis

The classification module and emerging threat module perform different functions.

Classification determines **what predefined threat category a document belongs to**, whereas emerging threat analysis attempts to identify **new or increasingly prominent patterns over time**.

Therefore, a topic can be considered important for emerging-threat analysis even when it does not correspond directly to a newly created classification category.

4.4 Topic Modeling Module

The topic modeling module discovers latent thematic structures in the CTI corpus.

Unlike supervised classification, topic modeling does not require predefined topic labels. It attempts to identify groups of terms that frequently occur together across documents.

The proposed framework can use:

- Latent Dirichlet Allocation (LDA)
- Non-Negative Matrix Factorization (NMF)

4.4.1 LDA-Based Topic Modeling

LDA represents documents as mixtures of topics and topics as probability distributions over words.

For a document d , the topic distribution can be represented as:

$$\theta_d = P(z|d)$$

where z represents a latent topic.

Similarly, each topic can be represented by a word distribution:

$$\phi_k = P(w|z_k)$$

where:

- w represents a word,
- z_k represents topic k .

The highest-probability words associated with each topic are extracted to interpret the semantic meaning of the topic.

4.4.2 NMF-Based Topic Modeling

NMF decomposes a non-negative document-term matrix into two lower-dimensional matrices.

Let X represent the document-term matrix:

$$X \approx WH$$

where:

- W represents document-topic weights,
- H represents topic-term weights.

The resulting topic-term matrix can be used to identify the most representative words associated with each topic.

4.4.3 Topic Identification

For each discovered topic, the framework extracts the most representative terms.

An example output is:

Topic	Representative Terms	Possible Interpretation
Topic 1	ransomware, encryption, file, ransom	Ransomware activity
Topic 2	phishing, email, credential, link	Phishing activity
Topic 3	vulnerability, exploit, patch, CVE	Vulnerability exploitation
Topic 4	botnet, command, server, infected	Botnet activity

The interpretations shown in such a table are analytical interpretations of the discovered terms and should be validated against the underlying documents.

4.4.4 Topic Number Selection

The number of topics is an important hyperparameter.

Different values of K can be evaluated using measures such as:

- Topic coherence
- Topic diversity
- Interpretability
- Temporal stability
- Domain relevance

The selected number of topics should therefore be justified using both quantitative and qualitative analysis.

4.5 Temporal Analysis

Temporal analysis examines how CTI topics change over time.

The primary objective is to determine whether particular threat-related topics:

- Appear for the first time
- Increase in prevalence
- Decrease in prevalence
- Persist over multiple periods
- Reappear after periods of low activity
- Develop new associated terminology

4.5.1 Temporal Grouping

Each CTI document is associated with a date.

The dataset can be divided into temporal intervals such as:

- Daily
- Weekly
- Monthly
- Quarterly

The selected interval depends on dataset size and temporal coverage.

For an interval t , the prevalence of topic k can be represented as:

$$P(k,t)$$

$$\frac{\sum_{d \in D_t} \theta_{d,k}}{|D_t|}$$

where:

- D_t represents documents belonging to time interval t ,
- $\theta_{d,k}$ represents the contribution of topic k to document d ,
- $P(k,t)$ represents the average prevalence of topic k during interval t .

4.5.2 Topic Evolution

The change in topic prevalence between consecutive periods can be calculated as:

$$\Delta P_k(t)$$

$$P(k,t) - P(k,t-1)$$

A positive value indicates an increase in topic prevalence, while a negative value indicates a decrease.

A normalized growth rate can also be calculated as:

\$\$ G_k(t)

$$\frac{P(k,t)-P(k,t-1)}{P(k,t-1)+\epsilon}$$

where ϵ is a small positive constant used to avoid division by zero.

4.5.3 Temporal Topic Matrix

The temporal topic distribution can be organized into a matrix:

Time Period	Topic 1	Topic 2	Topic 3	Topic 4
Period 1	0.18	0.25	0.31	0.26
Period 2	0.21	0.23	0.29	0.27
Period 3	0.34	0.19	0.25	0.22
Period 4	0.42	0.17	0.22	0.19

Such a representation allows changes in threat-related topics to be examined systematically.

4.5.4 Temporal Visualization

The temporal analysis can be visualized using:

- Topic prevalence line charts
- Topic heatmaps
- Time-series plots
- Topic evolution graphs
- Word-frequency trends

These visualizations help identify periods of unusual increases in particular threat-related themes.

4.6 Emerging Threat Scoring

The emerging threat scoring module is used to prioritize topics that demonstrate characteristics associated with emerging threat activity.

An important distinction is maintained between an **emerging threat signal** and a **confirmed new or zero-day threat**. A high score indicates that a topic satisfies selected analytical criteria; it does not independently establish that the underlying activity represents a confirmed security incident or zero-day vulnerability.

4.6.1 Scoring Factors

The proposed scoring mechanism considers multiple signals:

1. **Novelty**
2. **Growth**
3. **Persistence**
4. **Activity**
5. **Term Change**

These factors are intended to capture different dimensions of emerging behavior.

4.6.2 Novelty Score

Novelty represents the extent to which a topic is newly appearing in the observed dataset.

A simple binary representation can be defined as:

$$N_k(t) = \begin{cases} 1, & \text{if topic } k \text{ is newly observed at } t \\ 0, & \text{otherwise} \end{cases}$$

More detailed novelty measures can be developed using the appearance of new topic terms or changes in topic distributions.

4.6.3 Growth Score

The growth component measures the increase in topic prevalence:

$$G_k(t)$$

$$\frac{P(k,t) - P(k,t-1)}{P(k,t-1) + \epsilon}$$

Large positive values indicate rapid increases relative to the previous period.

Because raw growth values may have different ranges, they can be normalized before combining them with other components.

4.6.4 Persistence Score

Persistence measures whether a topic remains active over multiple consecutive time periods.

For example:

$$S_k(t) = \frac{C_k(t)}{W}$$

where:

- $C_k(t)$ is the number of recent time windows in which topic k remained active,
- W is the total number of considered windows.

A persistent topic receives a higher score than a topic that appears only once.

4.6.5 Activity Score

The activity component represents the relative amount of CTI information associated with a topic.

It can be estimated using the number or proportion of documents assigned strongly to the topic during a given time period.

4.6.6 Term Change Score

The framework can also examine changes in important terms associated with a topic.

The appearance of previously uncommon cybersecurity terms, malware names, attack techniques, vulnerabilities, or infrastructure-related terminology can provide additional evidence for a changing topic.

4.6.7 Combined Emerging Threat Score

The individual components can be combined into a normalized score:

$ETS_k(t)$

$$w_{NN}k(t) + w_{GG}k(t) + w_{SS}k(t) + w_{AA}k(t) + w_{TT}k(t)$$

where:

- $ETS_k(t)$ is the Emerging Threat Score,
- $N_k(t)$ is the novelty score,
- $G_k(t)$ is the growth score,
- $S_k(t)$ is the persistence score,
- $A_k(t)$ is the activity score,
- $T_k(t)$ is the term-change score,
- w_N, w_G, w_S, w_A, w_T are weighting coefficients.

The weights satisfy:

$$w_N + w_G + w_S + w_A + w_T = 1$$

The weights can initially be selected based on the experimental design and subsequently examined through sensitivity analysis.

4.6.8 Emerging Threat Signal

A topic may be flagged for further investigation when its score exceeds a selected threshold:

$$ETS_k(t) \geq \tau$$

where τ represents the selected threshold.

The threshold should be determined experimentally rather than arbitrarily. The resulting output should be interpreted as a **candidate emerging-threat signal** requiring further security validation.

An example output is:

Topic	Time Period	Growth	Persistence	Novelty	ETS	Status
Topic 3	Period 2	0.21	0.70	0.80	0.61	Candidate
Topic 7	Period 3	0.48	0.85	0.75	0.79	Candidate
Topic 2	Period 4	0.05	0.30	0.10	0.12	Low Signal

The values in this table are illustrative and will be replaced by experimentally obtained values in Chapter 5.

4.7 Implementation

The proposed framework is implemented using a Python-based data science environment. The implementation is divided into data handling, preprocessing, machine learning, topic modeling, temporal analysis, scoring, and visualization components.

4.7.1 Software Environment

The implementation can use the following software components:

Component	Technology
Programming Language	Python
Development Environment	Jupyter Notebook
Data Processing	Pandas, NumPy
Text Processing	NLTK / spaCy
Feature Extraction	scikit-learn
Machine Learning	scikit-learn
Topic Modeling	scikit-learn / Gensim
Visualization	Matplotlib / Seaborn
Data Storage	CSV / JSON / SQLite
Version Control	Git

The exact libraries and versions used for the final experiments should be documented to support reproducibility.

4.7.2 Implementation Workflow

The implementation follows the sequence:

1. Load CTI Dataset
↓
2. Validate Dataset
↓
3. Clean and Preprocess Text
↓
4. Generate TF-IDF Features
↓
5. Split Dataset
↓
6. Train Classification Models
↓
7. Evaluate Classification
↓
8. Build Topic Modeling Corpus
↓
9. Train LDA / NMF Models
↓
10. Extract Topic Distributions
↓
11. Group Topics by Time
↓
12. Calculate Temporal Changes
↓
13. Calculate Emerging Threat Scores
↓
14. Generate Visualizations
↓
15. Store and Analyze Results

4.7.3 Dataset Loading

The dataset is loaded into a structured data frame.

A simplified implementation is:

```
import pandas as pd

df = pd.read_csv("cti_reports.csv")

print(df.shape)
print(df.columns)
print(df.head())
```

The dataset is then checked for missing values and duplicate records.

```
print(df.isnull().sum())
print("Duplicate records:", df.duplicated().sum())
```

4.7.4 Text Preprocessing

A preprocessing function is applied to the textual data.

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"^[a-zA-Z0-9\s\-\._:/]", " ", text)
    return text.strip()

df["clean_text"] = df["text"].apply(clean_text)
```

The final preprocessing implementation may be extended with tokenization, stop-word removal, lemmatization, and cybersecurity-specific normalization according to the selected dataset.

4.7.5 TF-IDF Feature Extraction

TF-IDF features are generated using scikit-learn.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1, 2),
    min_df=2
)

X = vectorizer.fit_transform(df["clean_text"])
```

The final hyperparameters should be selected through experimentation and documented in Chapter 5.

4.7.6 Classification Implementation

The classification models can be implemented using scikit-learn.

```
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC

X_train, X_test, y_train, y_test = train_test_split(
    X,
    df["category"],
    test_size=0.20,
    random_state=42,
    stratify=df["category"]
)

model = LinearSVC()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Other algorithms can be implemented using the same training and evaluation pipeline.

4.7.7 Topic Modeling Implementation

For NMF-based topic modeling:

```

from sklearn.decomposition import NMF

nmf = NMF(
    n_components=10,
    random_state=42
)

W = nmf.fit_transform(X)
H = nmf.components_

```

The most representative terms for each topic can then be extracted.

```

terms = vectorizer.get_feature_names_out()

for topic_idx, topic in enumerate(H):
    top_terms = topic.argsort()[-10:][::-1]
    words = [terms[i] for i in top_terms]
    print(f"Topic {topic_idx + 1}: {words}")

```

LDA can similarly be implemented using an appropriate document-term representation.

4.7.8 Temporal Topic Analysis

After obtaining document-topic representations, the topic scores are associated with the document dates.

```

df["date"] = pd.to_datetime(df["date"])

topic_columns = [
    f"topic_{i+1}"
    for i in range(W.shape[1])
]

topic_df = pd.DataFrame(
    W,
    columns=topic_columns,
    index=df.index
)

df = pd.concat([df, topic_df], axis=1)

```

The data can then be grouped by month or another selected time interval.

```

df["period"] = df["date"].dt.to_period("M")

temporal_topics = df.groupby("period")[topic_columns].mean()

```

This produces a temporal topic matrix suitable for trend analysis.

4.7.9 Emerging Threat Score Implementation

The temporal topic matrix can be used to calculate changes between consecutive periods.

```

topic_growth = temporal_topics.pct_change()

```

Additional components such as persistence, novelty, and activity can then be normalized and combined according to the scoring formulation defined in Section 4.6.

A conceptual implementation is:

```
emerging_score = (  
    w_n * novelty_score +  
    w_g * growth_score +  
    w_s * persistence_score +  
    w_a * activity_score +  
    w_t * term_change_score  
)
```

The final implementation should ensure that all component scores are normalized to comparable ranges before calculating the combined score.

4.7.10 Visualization

Visualization is used to support interpretation of classification and temporal results.

Important visualizations include:

- Class distribution plots
- Confusion matrices
- Precision-recall curves
- F1-score comparison
- Topic-word distributions
- Topic prevalence over time
- Temporal heatmaps
- Emerging threat score trends

For example, topic evolution can be visualized using a line plot:

```
import matplotlib.pyplot as plt  
  
temporal_topics.plot(figsize=(12, 6))  
plt.xlabel("Time Period")  
plt.ylabel("Average Topic Contribution")  
plt.title("Temporal Evolution of CTI Topics")  
plt.legend()  
plt.tight_layout()  
plt.show()
```

4.7.11 Result Storage

The framework stores intermediate and final results to support reproducibility.

Potential output files include:

```
results/
├── classification/
│   ├── classification_report.csv
│   ├── confusion_matrix.csv
│   └── model_comparison.csv
├── topics/
│   ├── topic_terms.csv
│   ├── document_topics.csv
│   └── topic_coherence.csv
├── temporal/
│   ├── temporal_topic_distribution.csv
│   └── topic_growth.csv
├── emerging/
│   ├── emerging_threat_scores.csv
│   └── candidate_emerging_topics.csv
└── figures/
    ├── class_distribution.png
    ├── confusion_matrix.png
    ├── topic_distribution.png
    └── temporal_trends.png
```

4.7.12 Reproducibility

To ensure reproducibility, the implementation records:

- Dataset version
- Dataset preprocessing procedure
- Feature extraction parameters
- Model parameters
- Topic number
- Random seed
- Train-test configuration
- Scoring weights
- Emerging threat threshold
- Python and library versions

Random states should be fixed where applicable so that experiments can be reproduced.

Chapter Summary

This chapter presented the proposed framework and its implementation for automated CTI analysis. The framework integrates data preprocessing, feature extraction, machine learning-based threat classification, topic modeling, temporal analysis, and emerging threat scoring.

The classification module identifies predefined threat categories, while the topic modeling module discovers latent threat-related themes. Temporal analysis tracks changes in these themes over time, and the emerging threat scoring mechanism combines novelty, growth, persistence, activity, and term-change signals to identify candidate emerging threat patterns.

The implementation provides a practical computational framework for evaluating the proposed research objectives. The experimental configuration, dataset distribution, model performance, topic quality, temporal trends, and emerging threat results are presented and analyzed in Chapter 5.